IMPERIAL

Digital VLSI Design Project

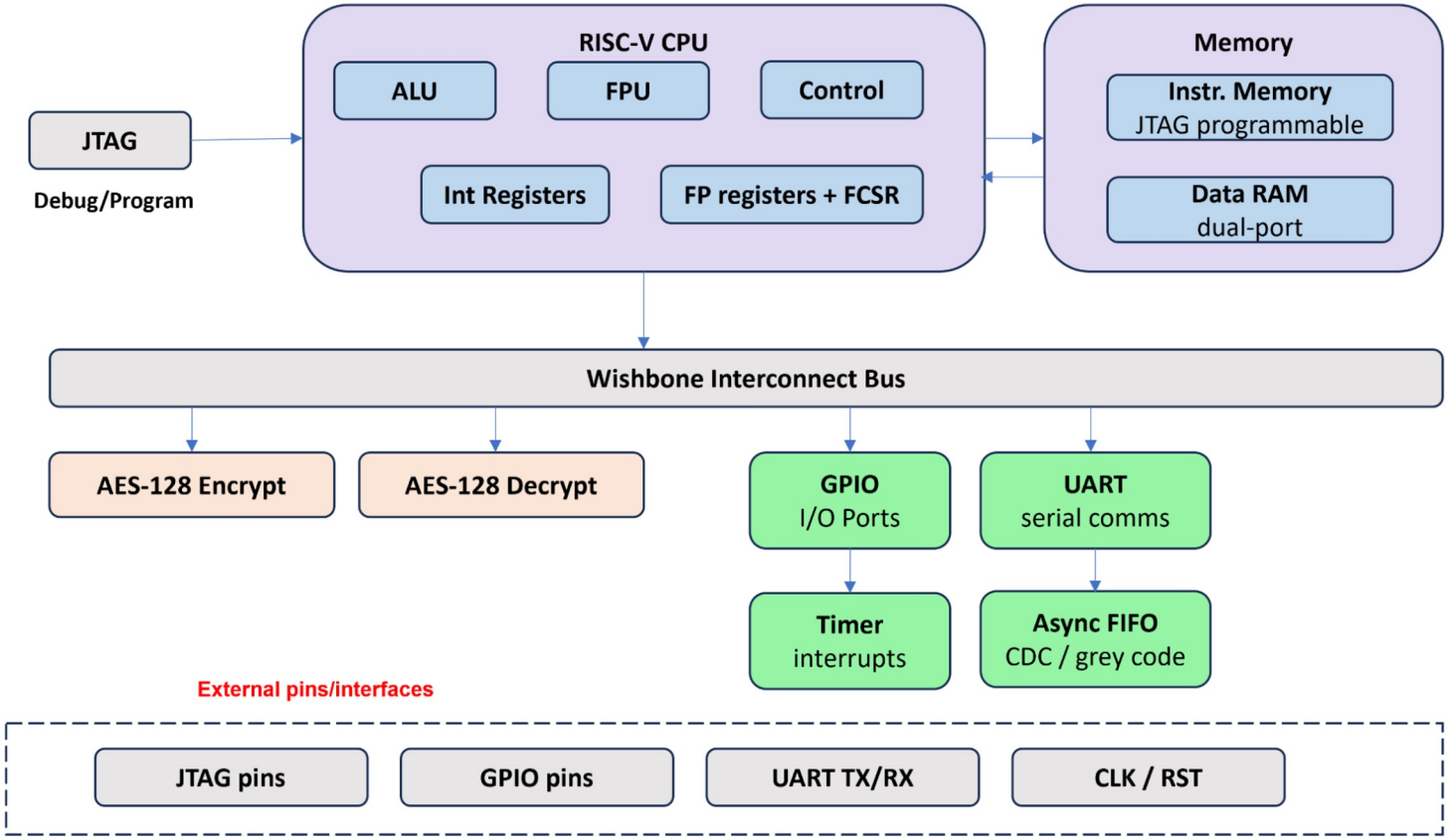
Team 2 Presentation - NAND Technologies

Chiedozie Ihebuzor, James Mitchell, Kiertan Solanki, Luke Hedley

System Overview and Motivation

The designed SoC implements **RISC-VIM processor** with the following extensions and peripherals:

- Single-precision floating point unit, compatible with the RISC F Extensio Specification
- AES Encryption Module
- AES Decryption Module
- JTAG Testing



Floating Point Unit Overview

The SoC contains a RISC-V floating point unit, compliant with both RISC-V and the IEEE-754 floating point specifications.

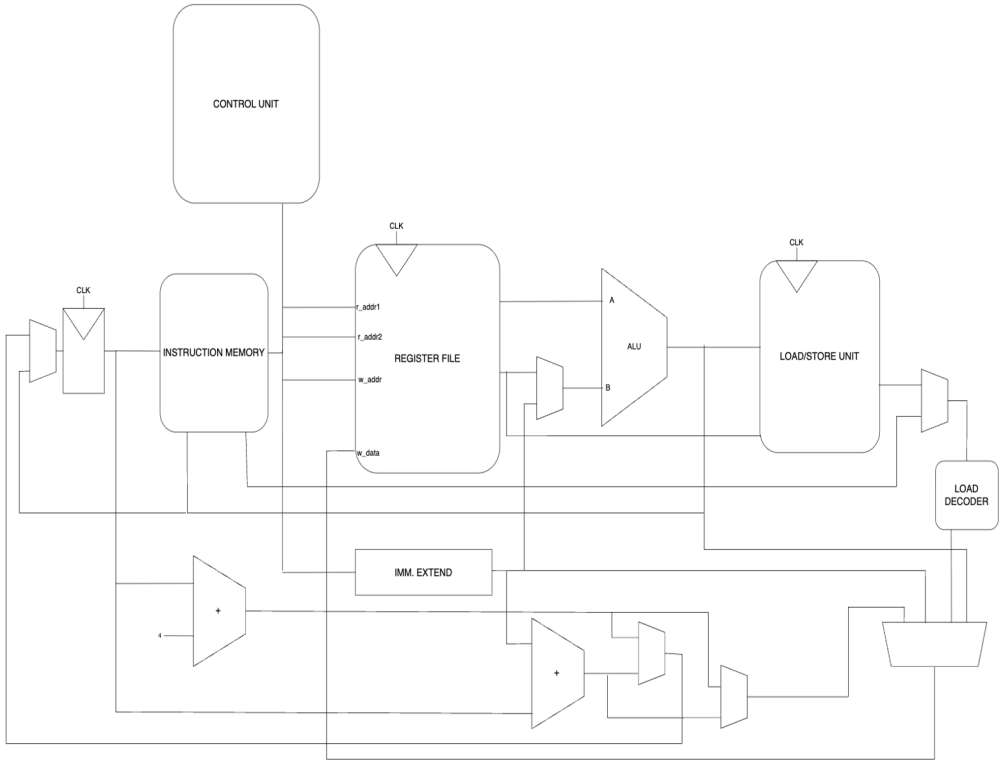
Full compiler support

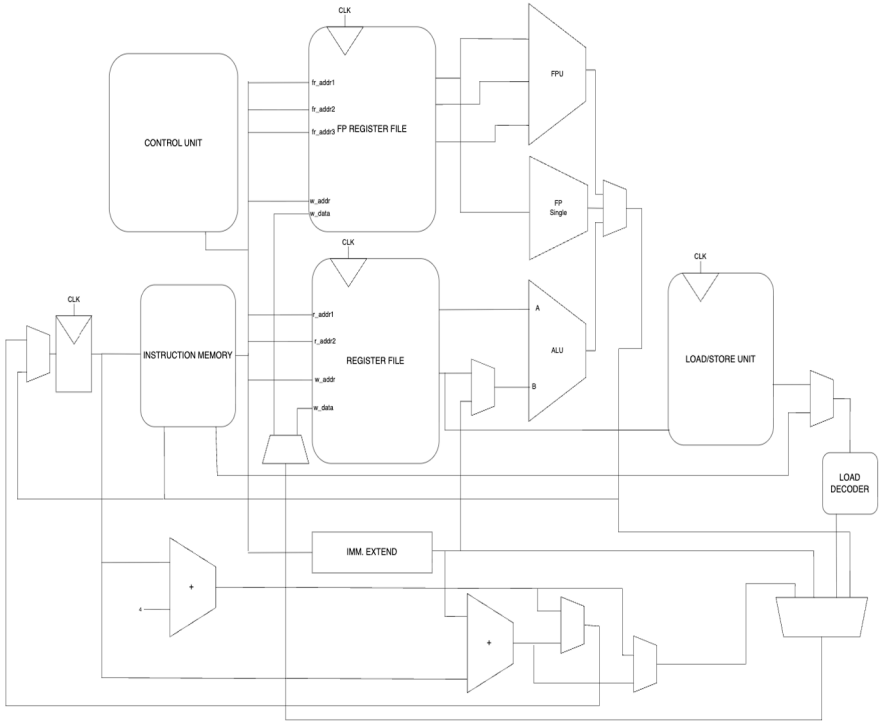
Modular, easy to turn off

Designed for space efficiency on the chip die

FPU Architecture

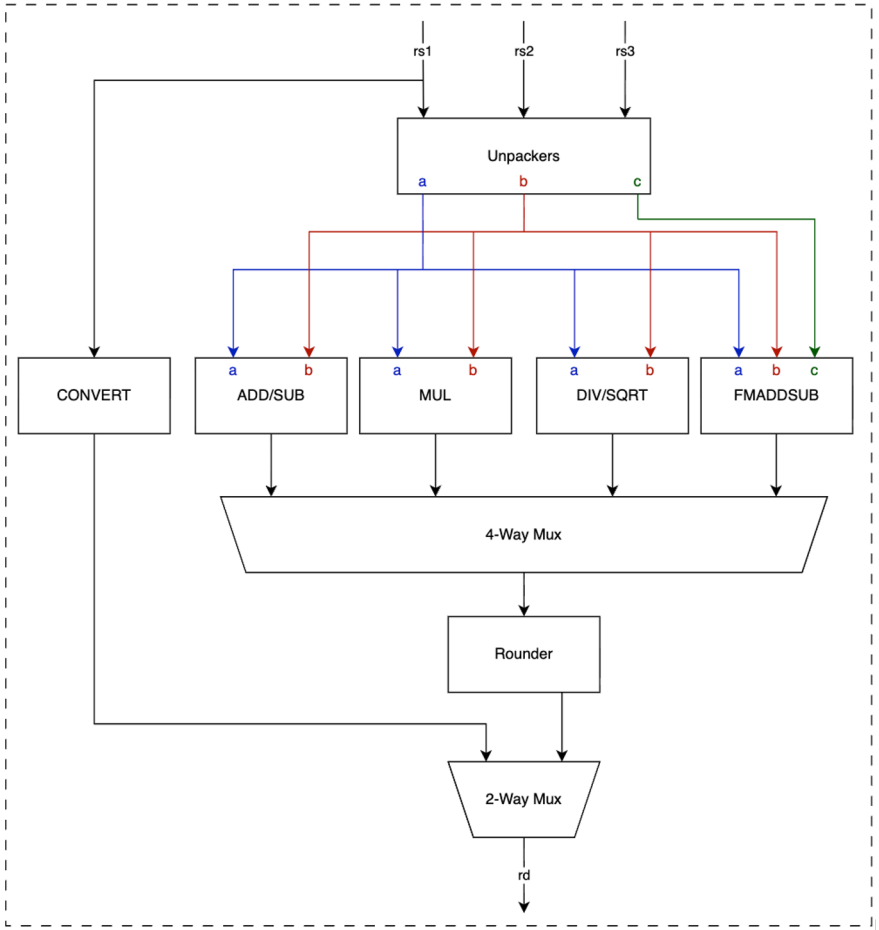
- Single and Multiple Cycle Domains
- Arithmetic and conversions are multi-cycle
- Logic operations are single cycle
- Decoder module that extends the CPU control unit
- Separate 32 unit floating point register file


CONTROL UNIT
CLK
INSTRUCTION MEMORY
CLK
r_addr1
r_addr2
w_addr
w_data
REGISTER FILE
A
B
ALU
CLK
LOAD/STORE UNIT
LOAD
DECODER
IMM. EXTEND
4
+
+


CONTROL UNIT
CLK
fr_addr1
fr_addr2
fr_addr3
FP REGISTER FILE
w_addr
w_data
FPU
FP
Single
CLK
INSTRUCTION MEMORY
CLK
r_addr1
r_addr2
w_addr
w_data
REGISTER FILE
A
B
ALU
CLK
LOAD/STORE UNIT
LOAD
DECODER
IMM. EXTEND
4
+
+

FPU Arithmetic

- Variable-length pipelines
- IDLE->BUSY->DONE state machine
- Custom intermediate representation for hardware reuse
- Rounding is fixed in the current implementation
- Support for flags and writing to CSR
- Newton-Raphson DIV/SQRT



FPU - Testing

- Fully modular, can enable/disable to determine error causes
- Test single and multi-cycle domains individually
- Easy to display output of floating point operations on the GPIO, using same method as main CPU
- Test operations individually then increase complexity of sequential operations

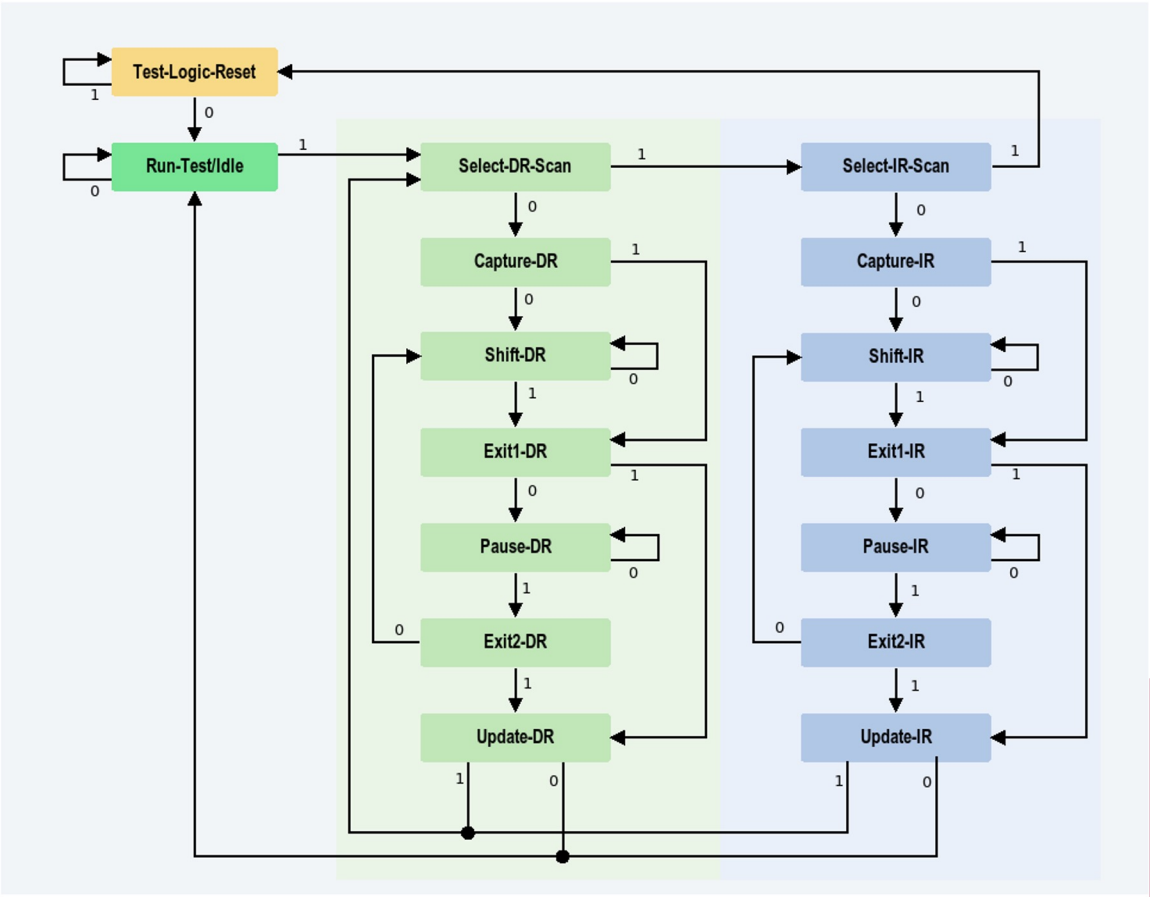
FPU - Further Work

- Combine all hardware to reduce area
- Add extra rounding mode support (at CPU level)
- Improve flag support and make CSR functional
- State machine improved to enable fully sequential arithmetic (currently mitigated at compiler level)

External Access - JTAG

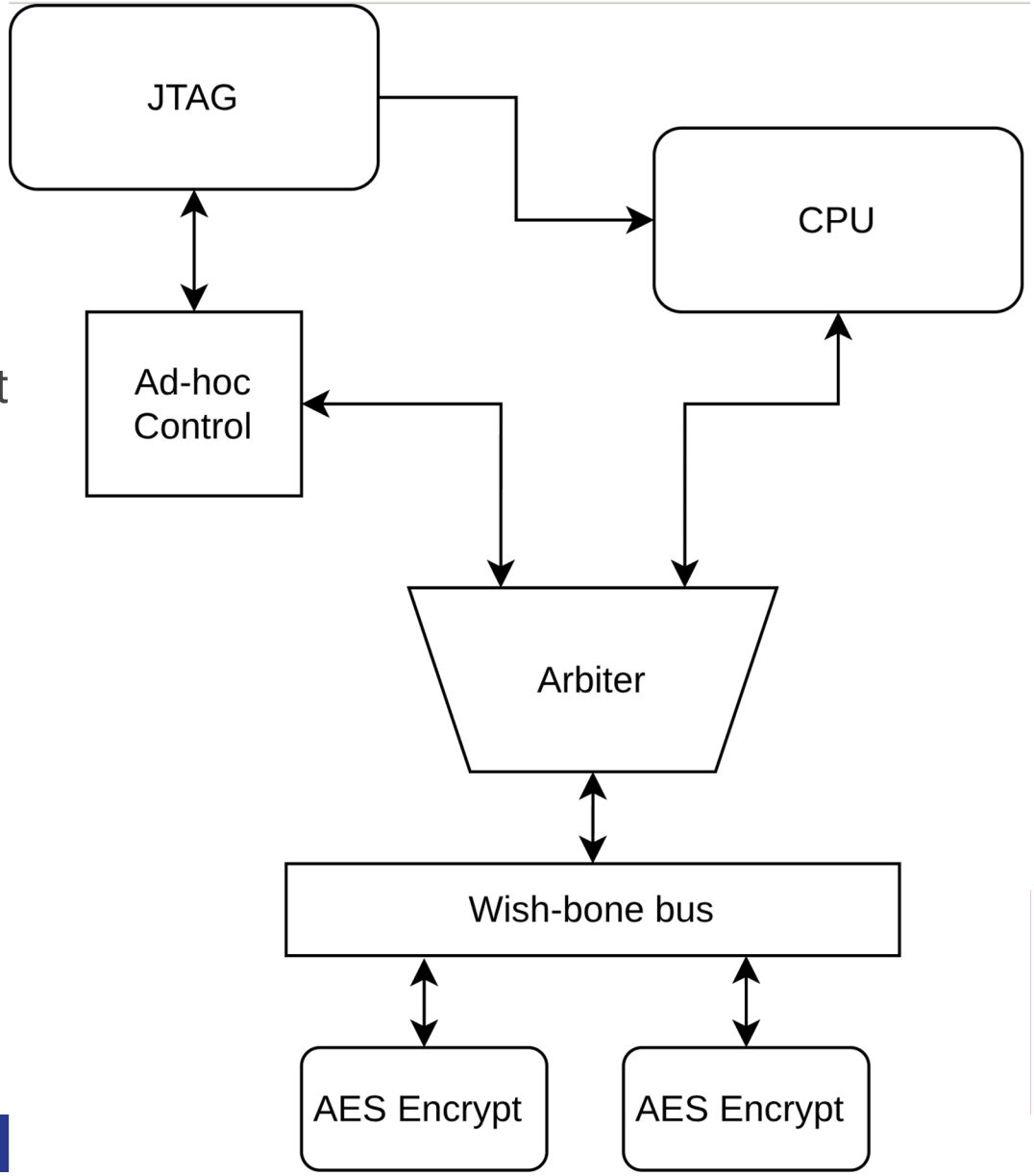
Test Access

- Ports: tck, tms, tdi, tdo
- TCK clocked by test controller
- Navigate state machine with TMS
- IR - Instruction Registers
- DR - Data Registers



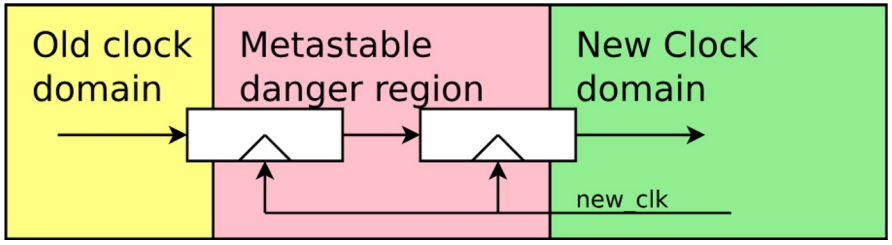
Ad-hoc test interface

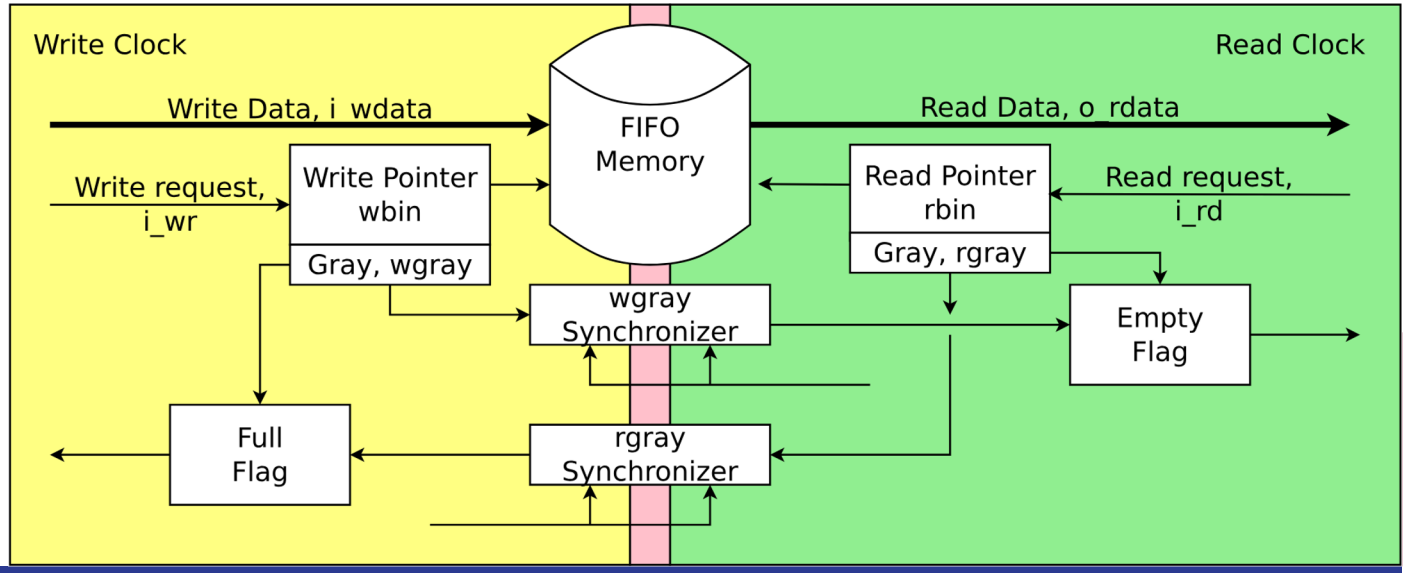
- Controllability, Observability, Survivabilit
- CPU bypass
- Control sequence
  - Set CPU to NO-OP
  - Latch JTAG mode to control
  - Set JTAG to wishbone write
  - Shift in address + data via tdi
  - Set JTAG to wishbone read req
  - Shift in req address
  - Set JTAG to wishbone read
  - Shift out data
- JTAG-first Arbiter



External Access - GPIO

- GPIO output pins
- Fast -> Slow Clock buffering
- CDC
  - Synchroniser
  - Grey-code
- CPU back pressure



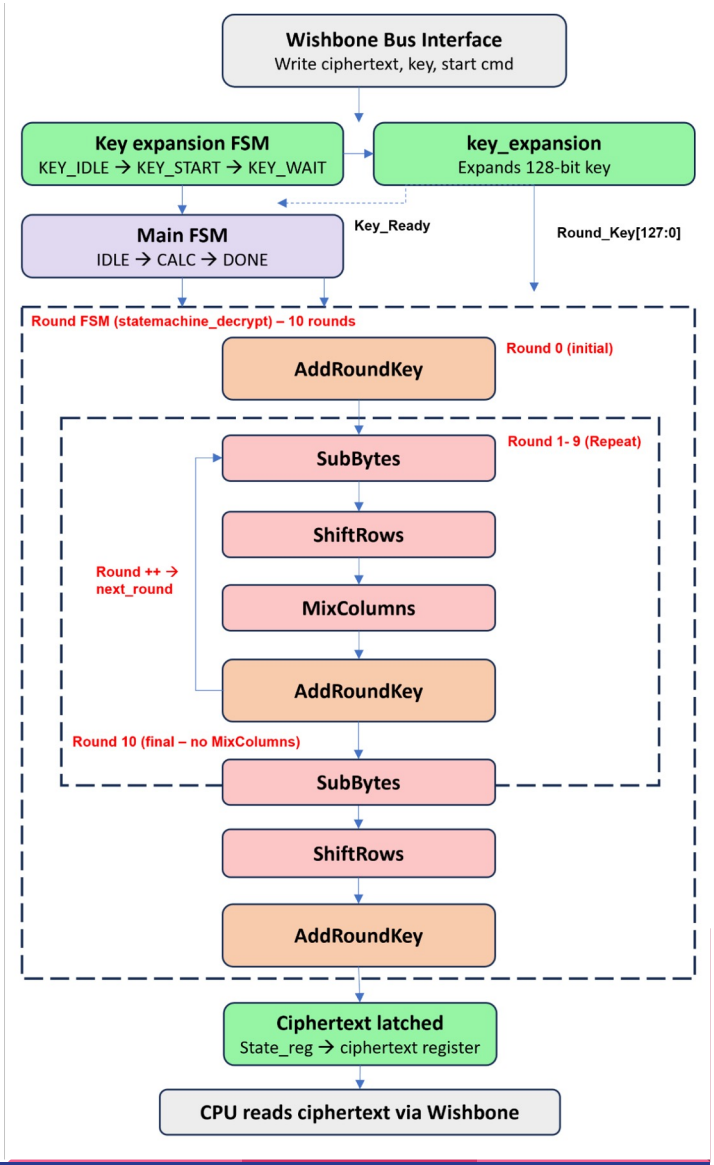


AES Encryption

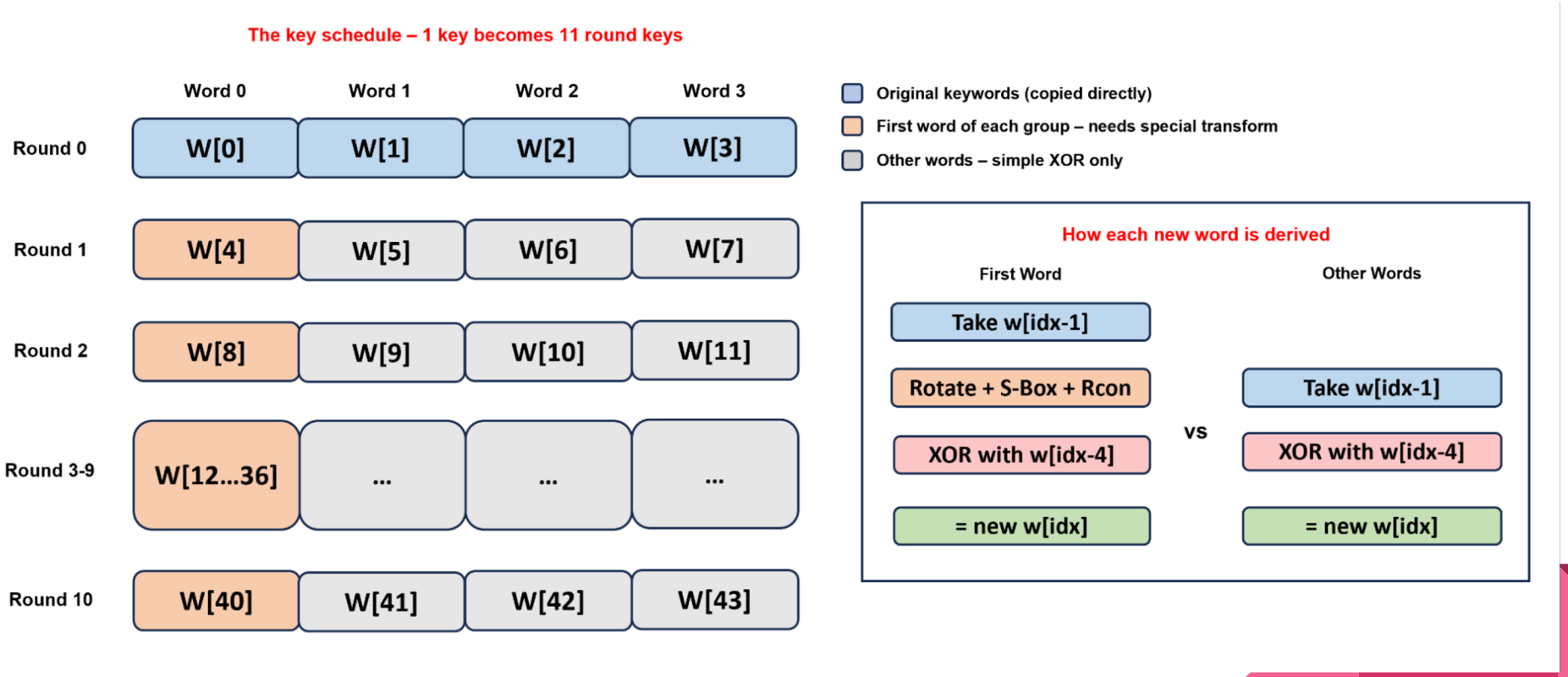
- **Advanced Encryption Standard (AES)** is a symmetric block cipher that processes data in 128-bit blocks.
- For AES-128, the key length is 128 bits, and the process consists of 10 rounds.

The internal state of the cipher is arranged as a 4x4 grid of bytes (128 bits total) - where column 1 is made of byte 0,1,2,3 and column 2 is byte 4,5,6,7, ect..

The encryption process transforms the plaintext into ciphertext through a series of substitution and permutation steps.



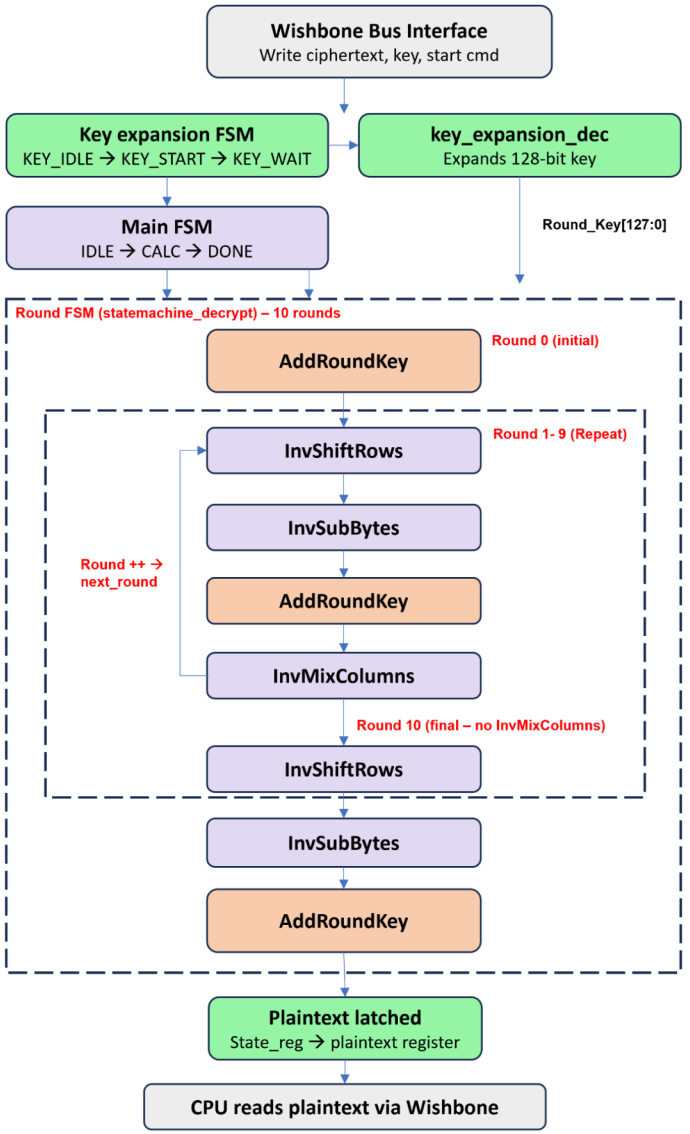
Key Schedule Implementation



AES Decryption

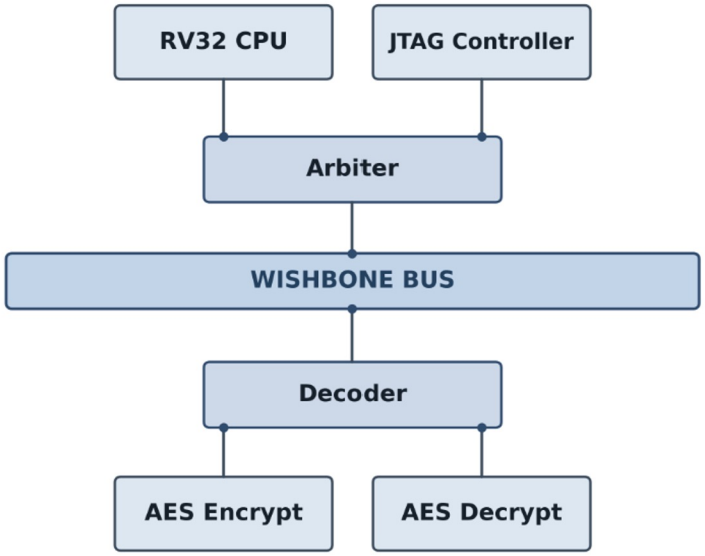
- **AES-128 decryption** process is the inverse of encryption.
- It takes the 128-bit ciphertext and the same 128-bit key used for encryption to recover the original plaintext.

The decryption process also consists of 10 rounds (for a 128-bit key), but the operations are applied in reverse order and using their inverse functions.



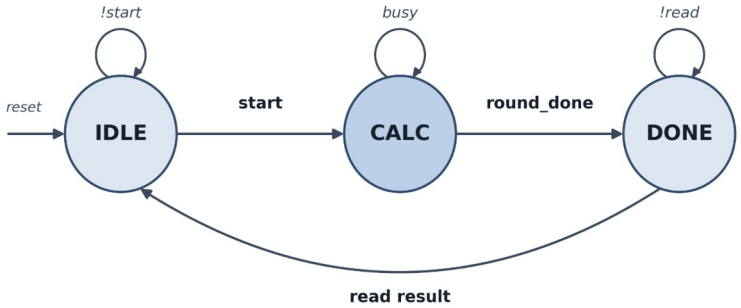
Peripheral Integration

- Accelerators are integrated as **memory-mapped Wishbone slaves**: CPU driven with ordinary **load/store**
- **Two masters one bus:** CPU + CTRL, merged by an arbiter
- **`ExtWishbone` = address decoder + read mux:** each slave has a base + 64 B window



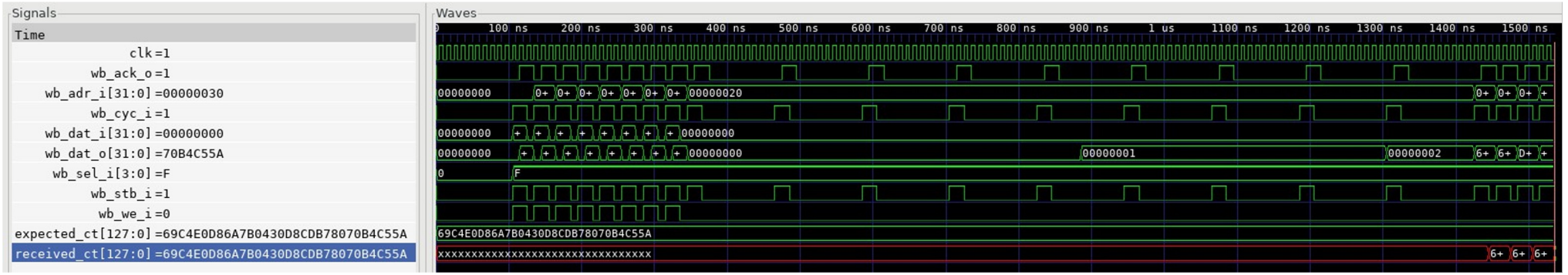
Inside a Peripheral

- Each accelerator is a set of registers on the Wishbone bus
- State Machine:
  - **IDLE**: takes in the writes
  - **CALC**: completes the 10 AES rounds, doesn't ack result reads until result is ready
  - **DONE**: result is ready, reads get acked
- CPU read doesn't finish until the result is done
- Design modular and reused in ModExp



Design Verification - AES Decryption and Encryption

Pre-synthesis & Post-synthesis Testing



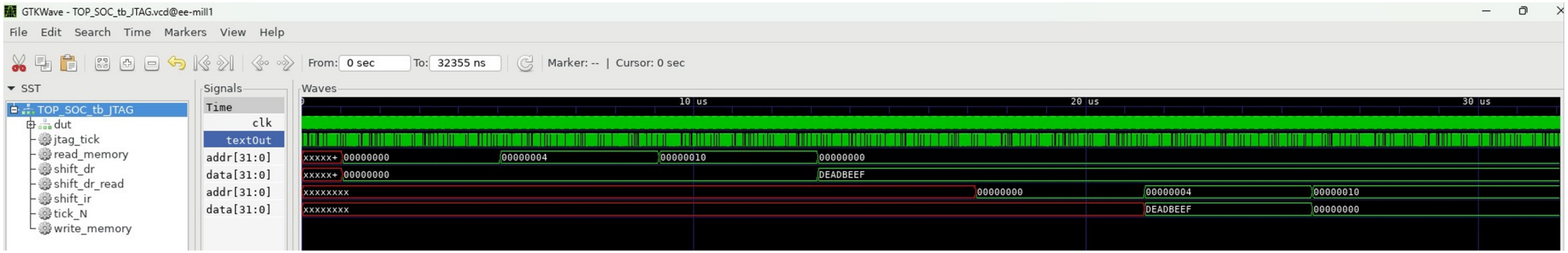

```
Starting AES-128 Encryption Test...
Plaintext written.
Writing last key word (triggers start)...
Status: IDLE...
Status: IDLE...
Status: IDLE...
Status: IDLE...
Status: IDLE...
Status: CALC...
Status: CALC...
Status: CALC...
Status: DONE!
Ciphertext Read Complete.
Expected: 69c4e0d86a7b0430d8cdb78070b4c55a
Received: 69c4e0d86a7b0430d8cdb78070b4c55a
TEST PASSED
Simulation complete via $finish(1) at time 1535 NS + 0
./SRC/aes128_tb.sv:172        $finish;
```

AES-128 Encryption Testbench

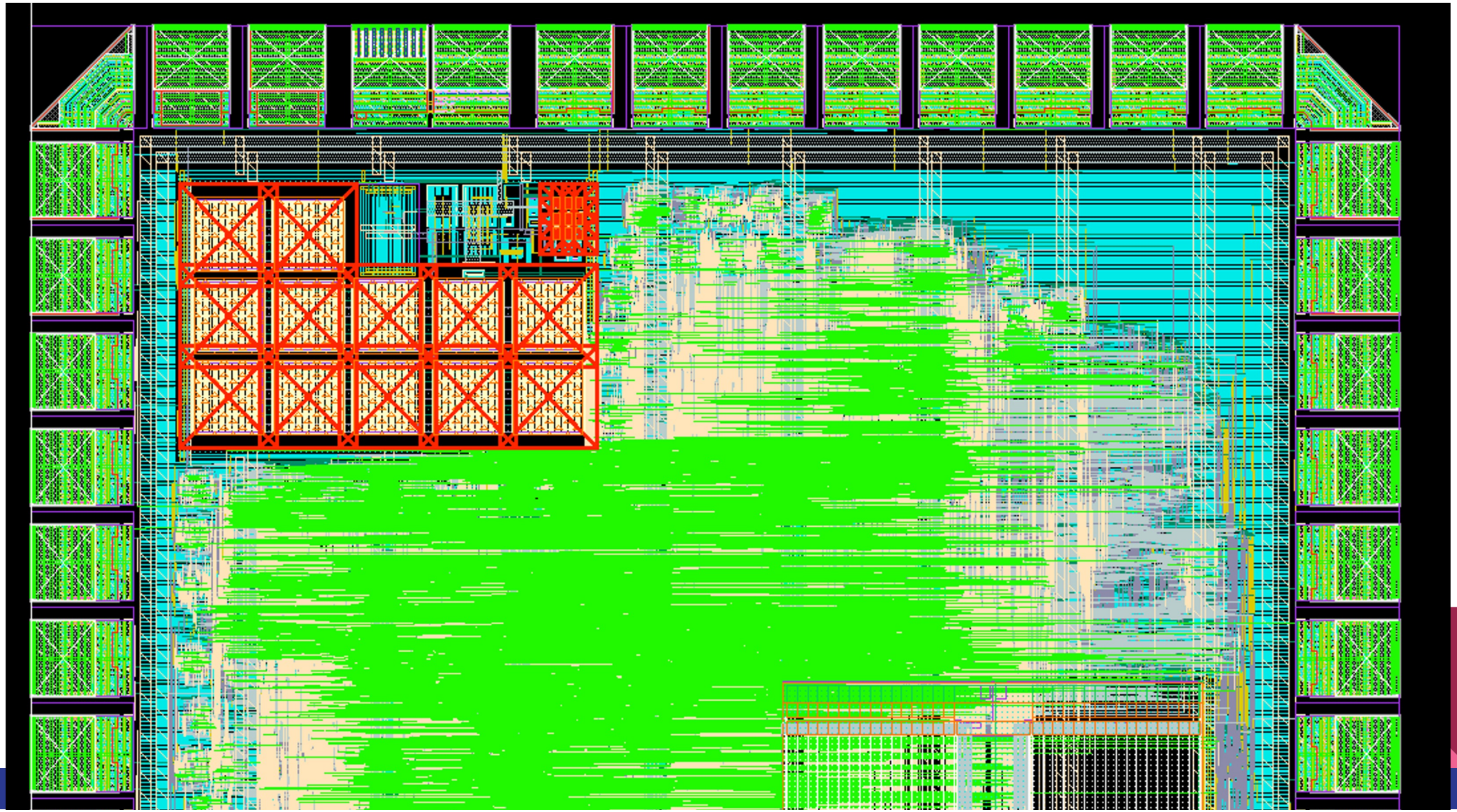
- Key: `128'h000102030405060708090a0b0c0d0e0f`
- Plaintext: `128'h00112233445566778899aabbccddeeff`
- Ciphertext should be `128'h69c4e0d86a7b0430d8cdb78070b4c55a`

Test Successfully Passed

Design Verification - JTAG Interface



Top trace - JTAG writes `DEADBEEF` to address `0x00000010` (after addressing `0x00000004`)
Bottom trace - readback at the same addresses confirms `DEADBEEF` returned correctly, validating the write



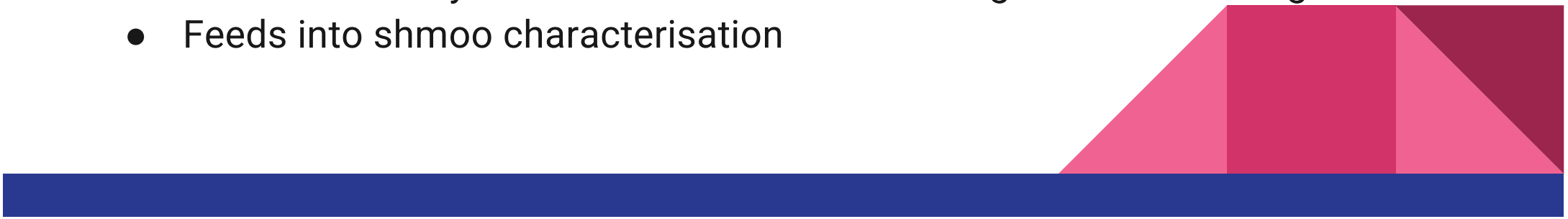
Silicon Debug

Power-on integrity

- Ramp V_DD slowly on a current-limited supply
- Measure quiescent supply current (I_DDQ) against the expected budget
  a. High I_DDQ: gate-oxide short, bridging fault, or V_DD-to-V_SS short
  b. Low / no current: open circuit or no activity

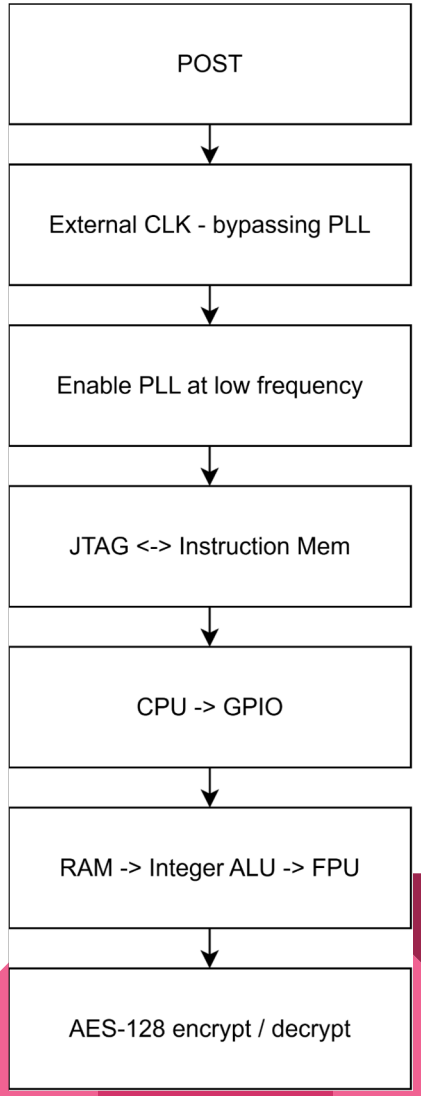
Voltage and temperature margining

- Confirm stability across the intended V_DD range before clocking
- Feeds into shmoo characterisation



8 Phase Bring-Up Strategy

- **POST -** Power on Self Test
- **External clock: bypass PLL.** Drive the chip from the FPGA
- **Enable PLL at low frequency.** Switch the clock source to the on-chip PLL (`clk_out`) at a conservative frequency.
- **JTAG ↔️ instruction memory.** Use the JTAG TAP to write and read back the instruction memory.
- **CPU → GPIO.** Load a small C program that writes known values to the GPIO pins and read them back.
- **RAM → Integer ALU → FPU:** load/store to RAM, then integer arithmetic, then floating-point.
- **AES-128 encrypt / decrypt.** Run the hardware AES engine against NIST test vectors.

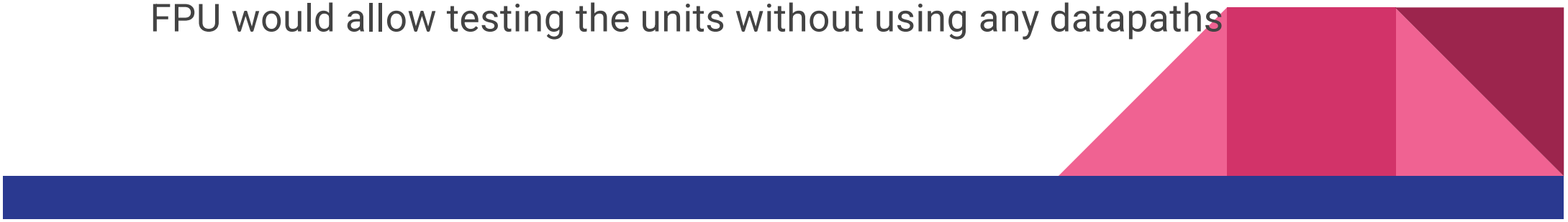


Project Evaluation - What Went Well

- **Strong team collaboration** - regular meetings, shared debugging, and parallel development with clean module interfaces
- **Wide technical scope delivered**: RISC-V CPU + FPU, AES-128 encrypt/decrypt with hardware key expansion, JTAG, async FIFO, full synthesis and place-and-route
- **Layered verification strategy** (unit, integration, JTAG) backed by CI/CD via GitHub Actions
- **Successfully passed all DRC (Design Rule Check) and LVS (Layout vs. Schematic) checks** - confirming the physical layout met TSMC 65nm foundry manufacturing rules and that the final layout correctly matched the synthesised netlist

Project Evaluation - Areas for Improvement

- **Post-synthesis verification -** due to time restrictions, only JTAG and CPU operations were extensively verified post-synthesis
  - Run more extensive post-synthesis simulation using Xcelium
- **Modular Exponentiation Accelerator -** required further RTL refinement to fully integrate this block into the system
- **Individual testing -** team members wrote and verified their own RTL, reducing independent cross-checking between design and verification
- **Built in Self Test** - A BIST unit to generate test vectors for the Peripherals and FPU would allow testing the units without using any datapaths



Acknowledgements

Thank you to Professor Peter Cheung, Luca Ridolfi, Peter Lui, and Apple Inc. for the continued support